

Kanishk Agarwal

Bangalore, India

+91-8777082770 — agarwalkanishk12345@gmail.com

[LinkedIn](#) — [GitHub](#) — [LeetCode](#) — [Portfolio](#)

Professional Summary

Backend Software Engineer with nearly 1.5 years of experience building **high-performance, concurrency-safe systems** and **AI-powered backend platforms**. Achieved **7 to 10 times latency improvement** (from 6 seconds to 0.8 seconds and 8 seconds to 0.7 seconds) through **event-driven architectures** and **asynchronous execution**. Improved correctness by eliminating **more than 90% invalid states** and achieving **more than 99% consistency**. Built **LLM-driven orchestration systems** across more than 20 microservices, reducing manual effort by **more than 40%** and task turnaround time by nearly **30%**.

Technical Skills

Languages: C++, C#, Python, SQL

Systems: Multithreading, Concurrency Control, Mutex, Condition Variables, Thread Pools, RAII, Rule of Five

Backend: .NET Core, REST APIs, Microservices, Event-Driven Architecture, WebSockets

Databases: MySQL, PostgreSQL, MongoDB, Indexing, Query Optimization, Caching, Execution Plans

AI Systems: LLM Systems, LangGraph, MCP, Retrieval-Augmented Generation, Vector Search

Tools: Docker, Git, CI/CD, AWS

Professional Experience

Software Development Engineer I
Harman International, Bangalore

Jan 2025 – Present

- Eliminated **more than 90% invalid booking states** by designing **lifecycle-driven invariants**, reducing manual correction effort by **more than 40%**.
- Ensured **consistency under concurrent workloads** by enforcing **transactional boundaries** and **locking mechanisms**, preventing conflicting updates across overlapping requests.
- Reduced booking response latency from **6 seconds to 0.8 seconds** by replacing polling-heavy API chains with **WebSocket-based event-driven architecture**.
- Reduced end-to-end workflow latency from **8 seconds to 0.7 seconds** by isolating **critical-path execution** and offloading non-blocking tasks to **asynchronous processing**.
- Prevented performance degradation under concurrent load using a **task-queue execution model**, eliminating request pile-ups and improving responsiveness.
- Reduced database query latency by nearly **25%** by adding **indexes on frequently queried fields** and rewriting **multi-join queries** to minimize full table scans.
- Reduced regression-related issues by more than **30%** by decoupling tightly coupled components and introducing **service-layer abstractions**.
- Improved system observability by surfacing third-party API failures, reducing issue diagnosis time by nearly **60%**.
- Handled **high-concurrency booking workflows** ensuring consistent system behavior under peak load conditions.
- Built **LLM-driven orchestration pipelines** using LangGraph and MCP for dynamic routing and execution across microservices.

Machine Learning Engineer Intern
Assisto Technologies Ltd., Pune

Jan 2024 – May 2024

- Designed and deployed **RAG pipelines** integrating LLM inference with Elasticsearch retrieval systems.
- Reduced inference latency by nearly **30%** through pipeline optimization.
- Implemented **vector-based semantic search** improving response relevance across queries.
- Integrated **speech processing systems** into production workflows.
- Containerized ML pipelines using Docker for reliable deployment.

Projects

Multithreaded Task Scheduler (C++)

- Designed and implemented a bounded **multithreaded task scheduler** with configurable thread pool, achieving nearly **3 to 5 times throughput improvement** over single-threaded execution.
- Built a **thread-safe producer-consumer queue** supporting more than **10,000 tasks per second** with stable latency under contention.
- Implemented **graceful shutdown mechanisms** ensuring zero task loss and safe completion of in-flight tasks.
- Reduced synchronization overhead by optimizing lock usage, improving CPU utilization by nearly **25 to 35 percent**.
- Validated correctness through stress testing with more than **100,000 tasks**, ensuring reliable concurrent behavior.

Custom STL-like Vector (C++)

- Designed and implemented a generic **vector** container with dynamic resizing and contiguous memory layout.
- Engineered **amortized constant time growth strategy** improving performance and cache locality.
- Implemented full **Rule of Five semantics** ensuring safe memory management and ownership handling.
- Optimized insertion and deletion using **manual memory management and placement new**.
- Built iterator support with **bounds-safe access** ensuring STL-like usability and correctness.

Education

B.Tech in Computer Science (AI Specialization)

2020 – 2024

MIT ADT University, Pune

CGPA: 8.0